

What are the principles, structures and operations that form the basis of database systems?

SYLLABUS CONTENT

By the end of this chapter, you should be able to:

- ▶ A3.1.1 Explain the features, benefits and limitations of a relational database

A3.1.1 Relational databases

■ Features

◆ **Database:** an organized collection of structured information or data that can be accessed in different ways.

◆ **Table:** a structure of rows and columns for storing a group of similar data.

◆ **Entity:** a living or non-living thing that can have data stored about it that can be described, e.g. a person, a chair or an aeroplane.

◆ **Tuple:** one instance of an entity; a row in a table.

◆ **Record:** one instance of an entity; a row in a table.

◆ **Attribute:** a data item or a characteristic of an entity; a column in a table.

A **database** refers to an organized collection of structured information or data that can be accessed in different ways. Typically, a database is stored electronically for fast retrieval and manipulation of data. There are different types of databases, but your focus will be on relational databases.

Relational databases have been predominantly used since 1980. Data in a relational database is organized as a set of tables made of columns and rows.

Tables

The **table** is used to describe entities. An **entity** refers to a living or non-living thing that can have data stored about it that can be described, such as a person, a chair or an aeroplane.

Each row, also called a **tuple**, will include a **record** or an instance of an entity, for example a specific person, a specific chair or a specific aeroplane. Each column, also called a “field” or an **attribute**, will include a data item or a characteristic of an entity, such as the age or the name of a person, the colour of a chair, the model of an aeroplane, and so on.

Consider the following example:

AEROPLANE

Model	Manufacturer	PhysicalClassEngine	NoOfEngines
Rockwell Commander 112	Rockwell	Piston	1
Airbus A319 Neo	Airbus	Jet	2
Boeing 747-100	Boeing	Jet	4
Boeing 777-8	Boeing	Jet	2
Airbus A400M Atlas	Airbus	Turboprop	4
Boeing 747-100	Boeing	Jet	4

In the example above, the table name is AEROPLANE (this is the entity) and there are six records or tuples (rows in the table, excluding the table heading) and four attributes or fields (columns).

Primary key

If you are to uniquely identify a record in this table, you should add an extra field. This is required, as all the given fields have repeating values, so they cannot be used to identify a record. It is possible for a company to have two aeroplanes of the same type, manufactured by the same company, with the same type of engine, and so on.

Therefore, by adding an extra field to uniquely identify each record, the table will look like this:

AEROPLANE

PlaneID	Model	Manufacturer	PhysicalClassEngine	NoOfEngines
A01	Rockwell Commander 112	Rockwell	Piston	1
A02	Airbus A319 Neo	Airbus	Jet	2
A03	Boeing 747-100	Boeing	Jet	4
A04	Boeing 777-8	Boeing	Jet	2
A05	Airbus A400M Atlas	Airbus	Turboprop	4
A06	Boeing 747-100	Boeing	Jet	4

◆ **Primary key:** a field that uniquely identifies a record in a table.

◆ **Foreign key:** an attribute in a table that refers to the primary key in another table.

The PlaneID field is unique for each record (it has no duplicates) and this is called a **primary key**.

Foreign key

Consider that you want to record data about specific flights at an airport. The AEROPLANE table only provides information about the planes. Therefore, you will need to create a new table to register the flight details.

FLIGHTS

FlightID	Departure	Destination	PlaneID	FlightDate	DeptTime	ArrivalTime
LG8903	LUX	OTP	A03	01/07/25	17:00	20:20
OS864	CAI	VIE	A04	15/01/25	16:45	19:20
GB961	LHR	ZRH	A03	25/02/25	8:40	11:35

In this table, the FlightID acts as a primary key.

The two tables, FLIGHTS and AEROPLANE, are building a relationship, as the PlaneID from the AEROPLANE table is used in the FLIGHTS table to identify the type of plane being used for a specific flight. However, the PlaneID is no longer a primary key in the FLIGHTS table, as it has repeating values; here it acts as a **foreign key**.

A foreign key is an attribute or a set of attributes in one table that refers to the primary key in another table.

Composite key

If you are to introduce a third table to register pilots on the flight, it might look like this:

PILOTFLIGHT

FlightID	PilotID
LG8903	P500
OS864	P104
GB961	P500

◆ **Composite key:** a set of attributes that form a primary key.

◆ **Relationship:** a relation established between different tables, where the foreign key in one table refers to the primary key in another table.

The `FlightID` links to the `FLIGHTS` table and the `PilotID` links to the `PILOTS` table (supposing there is a `PILOTS` table as well that records pilots' details). In the `PILOTFLIGHT` table, there is no primary key. A solution could be to use a **composite key**, formed from the two attributes `FlightID` and `PilotID`.

A composite key is a set of attributes that forms a primary key to provide a unique identifier for a table.

■ Relationships

A **relationship** is created when there is a logical association between two or more database tables, in which one table contains one or more foreign keys that reference the primary keys of the other tables. They enable relational databases to divide and store data in separate tables, while connecting their data items.

To ensure data is always accurate, accessible and consistent, relational databases follow certain integrity rules. For example, the referential integrity rule prevents users or applications from entering inconsistent data. It is a constraint that ensures that no table will contain values of a foreign key that are not matched to the corresponding primary key. In other words, it makes sure that a foreign key always refers to a record that exists in another table. By applying referential integrity constraints, the data stays consistent throughout operations such as insertion, deletion and modification of tuples.

There are several types of relationships:

- one-to-one (1:1)
- one-to-many (1:m)
- many-to-one (m:1)
- many-to-many (m:m).

One-to-one relationships

When there is a one-to-one relationship between two tables, that means that one record in a table is associated with exactly one record in another table: the primary key corresponds to one or no data in another table. For example, each staff member of a school has one single staff ID; each country has exactly one capital city; or a user on a social media platform has a single user profile. Those are very rare types of relationships, which you will not frequently encounter when dealing with databases.

One-to-many relationships

This is a frequently used type of relationship, and it refers to one record in a table being associated with one or more records in another table: the foreign key of one table references the primary key of another table. Examples of one-to-many relationships are where one teacher teaches many subjects; one tourist visits many countries; one person owns many properties; one person has many bank accounts.

Many-to-one relationships

Many-to-one relationships are similar to one-to-many relationships, but they differ in their directionality. The availability of the entity and the side of the relationship it is on determines whether it is a one-to-many or a many-to-one relationship. For example, if one teacher is teaching multiple subjects, the relationship between the teachers and the subjects is one-to-many, while the relationship between subjects and the teachers is many-to-one. Examples of many-to-one relationships are where many students enrol in a single course; many people work for a single company; there are many galaxies in the universe.

Many-to-many relationships

This type of relationship appears when multiple records in a table have a relation with multiple records in another table. Examples of many-to-many relationships are where many customers purchase many products; many actors act in many movies.

The problem with a many-to-many relationship is that a foreign key attribute can hold a single value and so it cannot handle the many references required.

To implement such relationships in relational databases you must introduce a linking entity. This means that two one-to-many relationships will be created: one between the first table and the linking table and another one between the second table and the linking table.

In the example above, when you wanted to connect the `FLIGHTS` table with the `PILOTS` table, a third table was introduced called `PILOTFLIGHT`. As such, a relationship of one-to-many was established between the `FLIGHTS` and `PILOTFLIGHT` tables and a relationship of one-to-many was established between the `PILOTS` and `PILOTFLIGHT` tables. This is done because a many-to-many relationship cannot be physically represented in a database.

■ Benefits of relational databases

Community support

Relational databases have been around since the 1970s, and this is the most widely accepted model for databases. Therefore, there are lots of online communities able to provide support and guidance in building, maintaining and troubleshooting them.

Concurrency control

Concurrency control is a crucial database management system (DBMS) component. It manages simultaneous operations without them conflicting with each other, and its purpose is to maintain data integrity, consistency and isolation when multiple users or applications access the database at the same time.

Data consistency

Data consistency refers to data remaining in a consistent state from start to finish, reinforcing data integrity. This means that all copies or instances of the data are the same across all systems and databases. In relational databases, each piece of data is stored in only one place, and all related data is stored together in the same table. This ensures all users have access to accurate and up-to-date information.

Data integrity

Data integrity refers to the accuracy, completeness and consistency of data throughout its lifecycle. It ensures the data hasn't been tampered with or altered in any unauthorized way. Data validation techniques can be used to ensure data integrity.

Data retrieval

The process of retrieving data from a relational database is efficient and flexible. SQL allows for complex queries to be written to retrieve exactly the data needed, using `SELECT` statements, `JOINS`, `WHERE` clauses, and more. Users can also create ad hoc queries to retrieve data without needing predefined reports or programs.

Reduced data duplication

Relational databases ensure that you have common fields to be used to link up tables and match records, without having to duplicate all the details several times. Identifying and removing duplicate data reduces the amount of storage needed to store the data.

Reduced redundancy

Data redundancy refers to storing the same data in multiple locations at the same time. This may lead to inconsistencies, partial updates and unnecessary duplications. Relational databases allow you to reduce redundancy by normalizing the database (organizing the data to be stored into several tables, creating relationships between them to avoid repeated groups of attributes, and correctly enforcing their dependencies; non-key attributes being independent).

Reliable transaction processing

A transaction refers to a sequence of actions performed on a database that is considered as a single unit (such as inserting, deleting, updating data in a table). A transaction is a unit of work, or a logical action, that is independent of other transactions and is performed on a database by a database management system. A transaction is either executed in full or it is not executed at all. Transactions ensure data integrity and reliability within relational databases.

Scalability

Database scalability refers to the ability of a database to handle increasing amounts of data, number of users and types of requests without sacrificing performance or availability.

Relational databases are vertically scalable, meaning that they support the idea of adding more resources (CPU, RAM, hard drive space) to existing systems, which is a cheaper, easier and faster approach to handling increases.

Security features

Relational databases increase security by controlling access to stored data, ensuring only authorized users can interact with the database. They allow the assignment of unique user accounts with specific permissions based on the users' roles and responsibilities. They allow different views of tables for different access rights.

■ Limitations of relational databases

Big data scalability issues

Relational databases can be more difficult to scale as the size and complexity of the data increases. The performance can drop when manipulating large data sets (horizontal scaling) or dealing with complex queries; joins between tables can be slow and indexing strategies can be difficult to optimize.

Design complexity

Relational databases require a lot of structure and planning to design the tables and the relationships between them in a way that fits correctly to the requirements.

Hierarchical data handling

Storing hierarchical data in relational databases is challenging due to the mismatch between the hierarchical structure and the tabular nature of relational databases. Even if this is done through a strategy such as an adjacency list model (where each record contains a reference to its parent record, forming a tree-like structure, such as an employee table having a field referencing the manager's ID for each employee), it is challenging to retrieve and traverse hierarchies, especially for large sets of data, or to reorder nodes and perform queries on the subtrees created.

Rigid schema

Relational databases have a predefined schema (structure of the data and how it will be stored in the database). Defining the schema can be challenging as it is not easy to predict the data structure of the database beforehand, and changing it later is complicated. When it comes to changing the database structure, updating the schema is time-consuming and complicated.

Object-relational impedance mismatch

Object-relational impedance mismatch refers to the difficulties encountered when relational databases are used by a program written in an object-oriented programming language. A major mismatch between relational databases and OOP languages is the data type differences. Relational models do not allow the use of by-reference attributes (pointers), while OOP languages embrace this behaviour. There is no clear way to translate all OOP concepts into relational databases or vice versa, such as there is no way to translate inheritance to a relational database concept.

Unstructured data handling

Unstructured data refers to a collection of data where one record differs from another record. Not being able to identify common fields or attributes for the records makes it impossible to design a schema for such data (to represent them as relational databases).

Common mistake

When asked to explain concepts such as benefits and limitations of relational databases within a given scenario, candidates often identify general benefits and limitations without making any connections to the given scenario. To gain full marks, the scenario must be taken into consideration, as well as the number of marks awarded for the respective question.

ACTIVITY

Self-management skills: Create plans to prepare for summative assessments – keep track of topics that have been covered and how well you mastered each of them. Identify what you can do to master any topics you found more challenging.

REVIEW QUESTIONS

- 1 Define the terms:
 - a “primary key”
 - b “foreign key”.
- 2 Explain what is meant by referential integrity.
- 3 Explain the one-to-one, one-to-many and many-to-many types of relationships.
- 4 Discuss the benefits and limitations of a relational database.
- 5 Define the term “database”.

SYLLABUS CONTENT

By the end of this chapter, you should be able to:

- ▶ A3.2.1 Describe database schema
- ▶ A3.2.2 Construct entity-relationship diagrams (ERDs)
- ▶ A3.2.3 Outline the different data types used in relational databases
- ▶ A3.2.4 Construct tables for relational databases
- ▶ A3.2.5 Explain the difference between normal forms
- ▶ A3.2.6 Construct a database normalized to 3NF for a range of real-life scenarios
- ▶ A3.2.7 Evaluate the need for denormalizing databases

A3.2.1 Database schema

◆ **Database schema:** an architecture showing how data is organized and how the relationship between data is managed.

◆ **Conceptual schema:** an abstract model describing the structure of the data without considering how it will physically be implemented.

Database schema is an architecture showing how data is organized and how the relationship between data is managed. It provides a logical view of the database.

There are different types of database schemas:

- **Conceptual schema:** An abstract model describing the structure of the data without considering how it will physically be implemented.
- **Logical schema:** A detailed design of the structure of the tables (fields and data types), relationships between tables and constraints.
- **Physical schema:** Represents the implementation of the logical schema into a specific DBMS (database management system), showing how data is stored, indexed or accessed.

The use of database schemas improves:

- **data organization:** it provides clear structure for storing and organizing the data
- **data security:** it defines user permissions and views to protect data
- **data integrity:** it uses rules and constraints to maintain data accuracy and consistency
- **performance:** through the use of queries
- **scalability:** it allows for changes to the database without disrupting current applications.

The DBMS controls the creation, maintenance and usage of a database and it mediates between the data-handling applications and the operating system. The DBMS offers features such as database queries, forms, reports and charts to display the data.

■ Conceptual schema

Conceptual schema is a high-level representation of the database, defining its structure and organization. It is an abstract model that hides details such as implementation of the data structures or physical storage. It defines the entities, attributes and relationships between entities.

A common method of implementing conceptual schema is by using entity-relationship diagrams (ERDs).